



BatchIt

Buy Together, Save Together

Mobile + REST API Application

Flutter (Dart) • Django REST Framework • PostgreSQL

FLUTTER

DJANGO

POSTGRESQL

JENKINS

Introduction & Problem Statement

What is BatchIt?

A **mobile-first collaborative bulk purchasing** platform. Consumers form *buying groups (batches)* to unlock wholesale prices on essential goods.

Customers — search & join batches

Providers — verified businesses listing products

Admins — verify providers, ensure trust

Inspired By

Groupon (2008) model + Pinduoduo (2015) adapted for **mobile-first Sub-Saharan African commodity markets**.

The Problem

High per-unit retail prices for individuals

No digital channel for group commitment purchases

Limited access to wholesale channels

No verification/trust for small vendors

BatchIt solves this

Key Features

Batch Lifecycle Management

Create → Discover → Join → Fill → Complete. Live **progress bar**. Auto status transitions (open/filled/expired).

Dual Authentication

Email/password + **6-digit code** via Brevo SMTP, plus **Google OAuth 2.0**. JWT: 60 min access, 7 day refresh.

Provider Verification Workflow

Business submits docs → Admin reviews → Approved/Rejected with in-app notification.

Batch Group Chat

Each batch has a **persistent group chat room**. REST polling (POST messages / GET history). Coordinate with buyers.

Map-Based Discovery

OpenStreetMap via `flutter_map`. View providers and batches by **geographic proximity**. Tap pins for detail.

In-App Notifications

Triggered on batch creation, join, fill, and provider approval. Unread badge count. Provider subscription model.



Object-Oriented Design — Concepts & Patterns Applied

Encapsulation

Data and behaviour bundled inside model classes. Batch exposes only clean methods to callers:

- `progress_ratio()` — computes fill percentage
- `can_join(qty)` — validates remaining capacity

Flutter services (`BatchService`, `ApiClient`) hide all HTTP internals from UI widgets.

Inheritance

Django `AbstractUser` extended by two concrete user types:

- `Customer` — phone, preferences, profile photo
- `Provider` — business name, GPS coords, verification status

Polymorphism

Same interface, different runtime behaviour:

- `BasePermission` → `IsCustomer` / `IsProvider` / `IsAdmin`
- `ModelSerializer` → `BatchSerializer` / `OrderSerializer`

Abstraction & Composition

Abstraction: Service layer hides REST API details behind typed methods (`fetchBatches()`, `joinBatch()`).

Composition: Batch owns one `BatchChatRoom` (1:1) and many `BatchParticipant` objects (1:N). Customer relates to Provider via a `Subscription` join object.

Design Patterns Applied

Creational Patterns (Flutter)

Singleton — `ApiClient` uses a private constructor and a static `_instance`; all six service classes share one HTTP client and one auth-token state.

Factory — Every model class exposes a `fromJson()` factory constructor; the service layer instantiates them directly from decoded API responses.

Structural Patterns (Flutter)

Facade — `ApiClient` hides raw `http.Client` calls; token injection, 401 retry, and error parsing are invisible to every caller.

Repository — `BatchService`, `AuthService`, and `ProviderService` isolate all HTTP details from the six `ChangeNotifier` providers.

Behavioural Patterns

Observer (*Flutter*) — Six `ChangeNotifier` subclasses drive reactive UI: `notifyListeners()` triggers `Consumer` and `context.watch()` rebuilds automatically.

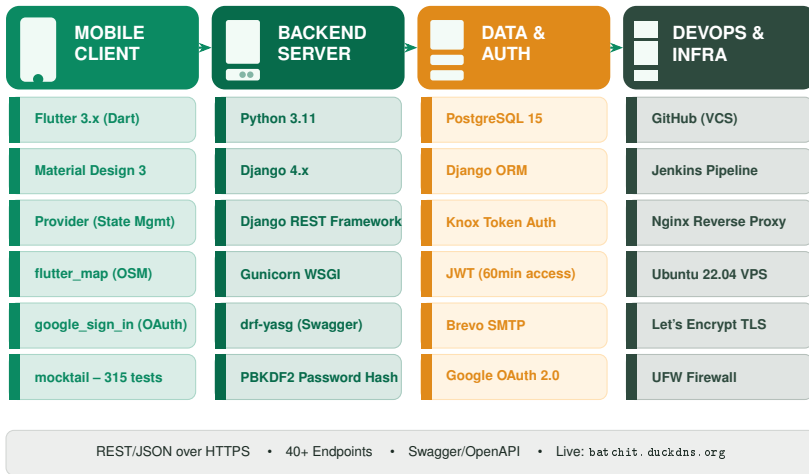
Strategy (*Django*) — DRF permission classes (`IsCustomer`, `IsProvider`, `IsAdmin`) and `ModelSerializer` subclasses are selected per endpoint at runtime.

Architectural Patterns (Django)

MVT — `models.py` defines schema; DRF `ListCreateAPIView` and `RetrieveUpdateDestroyAPIView` handle all REST logic without templates.

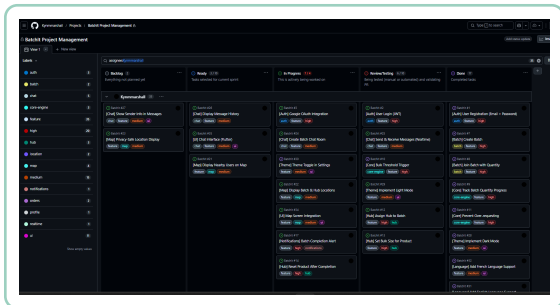
Serializer / Adapter — DRF serialisers map DB field names to the public API contract (e.g.

Technology Stack



Scrum Methodology & Team Organisation

4 Sprints • GitHub Projects Kanban • Feature Branches



Team Roles (5 Members)

- Kamdeu Yamdjeuson**
Scrum Master / DevOps & VPS Setup
- Njume Leslie**
Product Owner / Backend Lead
- Atsimbong Joshua**
Flutter Developer / UI & Maps
- Chijioke Emmanuel**
Backend Developer / Realtime & Chat
- Fru Chi**
Flutter Developer / Core Features

Conflicts Resolved

VPS cost refusal → members who declined to pay excluded from deployment credits

Docs funding → authorship withheld from non-contributing members

pubspec.yaml race condition → single owner assigned per sprint

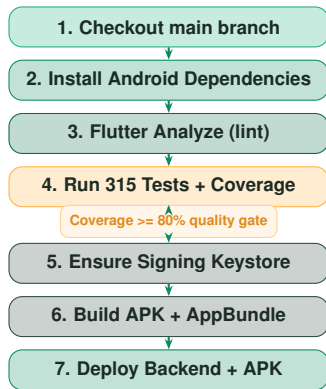
Migration conflicts → squash & rebase before each sprint start

API naming → RESTful convention enforced via DRF router

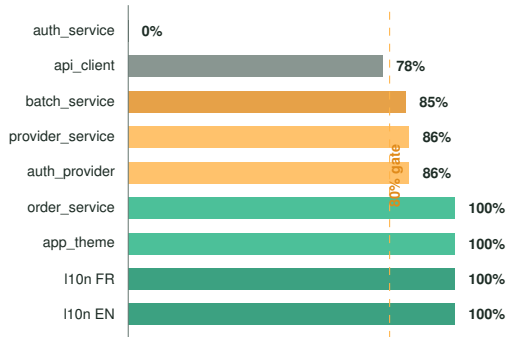


CI/CD Pipeline & Test Coverage

Jenkins Pipeline — Triggered on Every Push to `main`



Test Coverage — Build #46 (83.6% overall)



315 tests — all passing

83.6% line coverage (1466 / 1753 lines)

Packages: `mocktail` + `flutter_test`

Started at 28.3% — improved systematically

Results, Future Work & Conclusion

Achievements

40+

API Endpoints

315

Tests Passing

83.6%

Coverage

4 Sprints

Scrum delivery

14 min

CI/CD pipeline

- Full-stack Flutter + Django in 4 weeks
- Bilingual EN/FR Material Design 3 UI
- Production VPS: `batchit.duckdns.org`
- Signed APK auto-built via Jenkins
- 40+ documented REST API endpoints

Challenges Overcome

Jenkins keystore path • Google SHA-1 mismatch
Coverage 28% → 83.6% • JWT migration

Recommendations for Future Work

WS

Real-time chat with Django Channels + Redis

FCM

Firebase push notifications (Android + iOS)

PAY

MTN MoMo / Orange Money payment integration

GEO

PostGIS spatial filtering by GPS radius

iOS

iOS App Store deployment + Let's Encrypt HTTPS

WEB

React provider dashboard (web admin panel)

Thank You

Questions & Discussion Welcome

BatchIt — Buy Together, Save Together

SEN2241 • Group 14 • ICT University • Spring 2026

Instructor: Tekoh Palma Achu

GitHub: `github.com/Kynmmarshall/BatchIt`

Live: `batchit.duckdns.org`



Scan to visit `batchit.duckdns.org`